Tutorial A02

Workload Design

Nama	:	Muttaqin Muzakkir
NPM	:	2306207101
Mata Kuliah	:	Arsitektur Aplikasi Web (AAW)
Topik	:	Stream Processing (Kafka, RabbitMQ) & Advanced Database (MySQL Master-Slave)
Tanggal	:	9 Maret 2026

Section A: Apache Kafka

A1: Memulai Kafka dan Membuat Topic (Q1)

Kafka berjalan otomatis melalui Docker Compose (container **kafka** dengan port 9092). Berikut output pembuatan dan verifikasi topic *quickstart-events*:

```
$ docker compose up -d
 Container kafka  Started
 Container rabbitmq  Started
 Container mysql-master  Started
 Container mysql-slave  Started

$ docker ps --filter "name=kafka" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES    STATUS          PORTS
kafka    Up 17 seconds   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp

$ docker exec kafka /opt/kafka/bin/kafka-topics.sh \
    --create --topic quickstart-events --bootstrap-server localhost:9092
Created topic quickstart-events.

$ docker exec kafka /opt/kafka/bin/kafka-topics.sh \
    --list --bootstrap-server localhost:9092
quickstart-events

$ docker exec kafka /opt/kafka/bin/kafka-topics.sh \
    --describe --topic quickstart-events --bootstrap-server localhost:9092
Topic: quickstart-events  TopicId: kMt-7ZqETISWmsDejI2zgg
  PartitionCount: 1  ReplicationFactor: 1  Configs: min.insync.replicas=1
  Topic: quickstart-events  Partition: 0  Leader: 1  Replicas: 1  Isr: 1
```

Jawaban Q1:

Topic *quickstart-events* berhasil dibuat dan diverifikasi menggunakan perintah --list (menampilkan nama topic) dan --describe (menampilkan detail: PartitionCount=1, ReplicationFactor=1, Leader=1). Kafka berjalan dalam mode KRaft (tanpa ZooKeeper) di dalam container Docker pada port 9092.

A2: Producer Mengirim Events, Consumer Membaca (Q2 & Q3)

Producer mengirim 5 pesan ke topic, kemudian consumer membaca semua pesan dari awal (--from-beginning):

```
# Terminal Producer:
$ echo -e "Hello Kafka!\nEvent number 1\nEvent number 2\n
Event number 3\nStream processing test" | \
  docker exec -i kafka /opt/kafka/bin/kafka-console-producer.sh \
  --topic quickstart-events --bootstrap-server localhost:9092

# Terminal Consumer:
$ docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --topic quickstart-events --from-beginning --max-messages 10 \
  --bootstrap-server localhost:9092

Hello Kafka!
Event number 1
Event number 2
Event number 3
Stream processing test
Hello Kafka!
Event number 1
Event number 2
Event number 3
Stream processing test
Processed a total of 10 messages
```

Jawaban Q2:

Consumer menerima semua pesan yang dikirim producer secara real-time. Dengan flag --from-beginning, consumer juga membaca pesan lama yang sudah tersimpan di log Kafka. Hal ini menunjukkan bahwa Kafka bersifat persistent - pesan disimpan di disk dan dapat dibaca ulang kapan saja.

Jawaban Q3 (Jika Producer Membanjiri Data):

Jika producer mengirim data dalam jumlah sangat besar (flood), consumer akan mengalami *consumer lag* - yaitu perbedaan offset antara pesan terakhir yang diproduksi dan pesan terakhir yang dikonsumsi. Namun Kafka tidak kehilangan data karena pesan dibu&ffer; di disk (log retention). Consumer dapat catch-up ketika producer melambat. Inilah keunggulan Kafka sebagai message broker dengan durability tinggi.

A3: Dua Consumer Secara Bersamaan (Q4)

Dua consumer group berbeda (*consumer-group-a* dan *consumer-group-b*) membaca dari topic yang sama secara bersamaan:

```
# Consumer 1 (group-a):
$ docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --topic quickstart-events --from-beginning --max-messages 5 \
  --group consumer-group-a --bootstrap-server localhost:9092

Hello Kafka!
Event number 1
Event number 2
Event number 3
Stream processing test

# Consumer 2 (group-b) - berjalan bersamaan:
$ docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --topic quickstart-events --from-beginning --max-messages 5 \
  --group consumer-group-b --bootstrap-server localhost:9092

Hello Kafka!
Event number 1
Event number 2
Event number 3
Stream processing test
```

Jawaban Q4:

Tanpa consumer group (atau dengan group berbeda): kedua consumer menerima SEMUA pesan -setiap consumer mendapat salinan penuh dari topic. Ini cocok untuk use case broadcasting (misalnya logging + analytics membaca data yang sama). Dengan consumer group yang SAMA: pesan dibagi (partitioned) antar consumer dalam group -setiap pesan hanya diterima oleh satu consumer. Ini cocok untuk scaling horizontal pemrosesan pesan.

A4: Force-Stop Kafka, Observasi Behavior (Q5)

Kafka dihentikan paksa saat producer sedang berjalan, lalu di-restart untuk verifikasi durability:

```
# Stop kafka container:
$ docker stop kafka
kafka
Kafka stopped at: Mon Mar  9 16:50:46 WIB 2026

# Coba consumer saat kafka mati:
$ docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh ...
Error response from daemon: container 7a0f21ec... is not running

# Restart kafka:
```

```
$ docker start kafka
kafka

# Consumer membaca ulang - data tetap ada:
$ docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --topic quickstart-events --from-beginning --max-messages 10 \
  --bootstrap-server localhost:9092

Hello Kafka!
Event number 1
Event number 2
Event number 3
Stream processing test
Hello Kafka!
Event number 1
Event number 2
Event number 3
Stream processing test
Processed a total of 10 messages
```

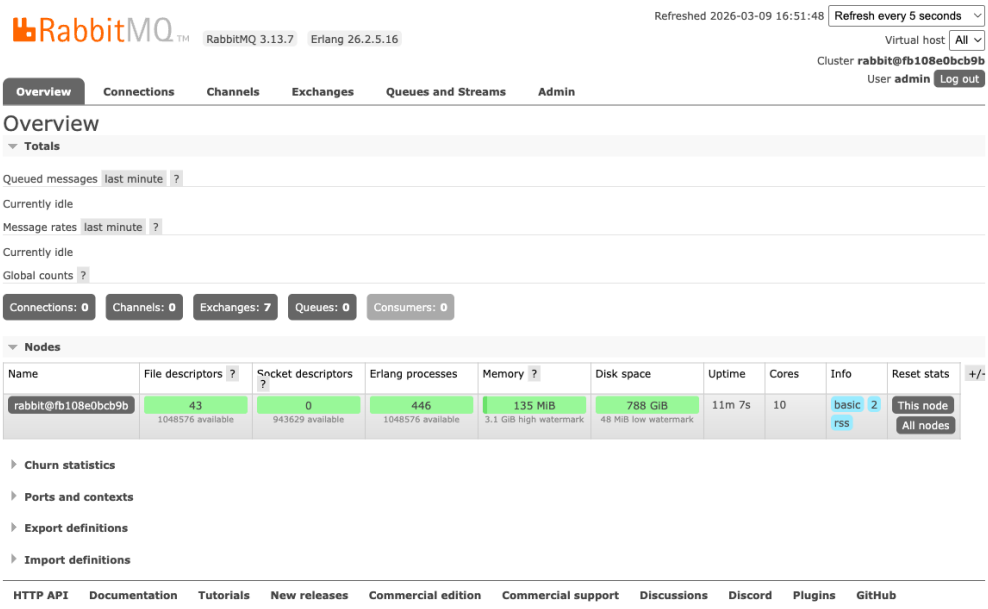
Jawaban Q5:

Ketika Kafka dihentikan paksa: (1) Producer mendapat error koneksi - pesan yang belum ter-commit ke broker hilang dari buffer producer. (2) Consumer tidak dapat terhubung dan berhenti menerima pesan. (3) Namun semua pesan yang sudah TERSIMPAN di Kafka log (disk) tetap aman - terbukti saat Kafka direstart, consumer dapat membaca ulang semua pesan dari awal. Kafka menggunakan write-ahead log yang durable, sehingga data tidak hilang ketika broker restart normal maupun paksa.

Section B: RabbitMQ

B1: RabbitMQ Management Dashboard (Q1)

Dashboard diakses melalui browser di http://localhost:15672 dengan kredensial **admin/admin**:



Gambar B1: RabbitMQ Management Dashboard - Overview tab

Jawaban Q1:

Dashboard RabbitMQ Management menampilkan: (1) **Totals** - jumlah pesan dalam antrian, message rates (publish/deliver/ack per detik), dan global counts (Connections, Channels, Exchanges, Queues, Consumers). (2) **Nodes** - health node termasuk file descriptors, memory usage (135 MiB), disk space, dan uptime. (3) **Tabs navigasi** untuk monitoring Connections, Channels, Exchanges, Queues secara detail. Dashboard ini memudahkan observasi real-time terhadap aliran pesan di sistem.

B2: publish.py dan consume.py

publish.py:

```
import pika

credentials = pika.PlainCredentials('admin', 'admin')
connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost', credentials=credentials)
)
channel = connection.channel()

channel.queue_declare(queue='hello')

channel.basic_publish(exchange='', routing_key='hello', body='Hello RabbitMQ!')
print("[x] Sent 'Hello RabbitMQ!'")

connection.close()
```

consume.py:

```
import pika

credentials = pika.PlainCredentials('admin', 'admin')
connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost', credentials=credentials)
)
channel = connection.channel()

channel.queue_declare(queue='hello')

def callback(ch, method, properties, body):
    print(f"[x] Received {body.decode()}")

channel.basic_consume(queue='hello', on_message_callback=callback, auto_ack=True)
print('[*] Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

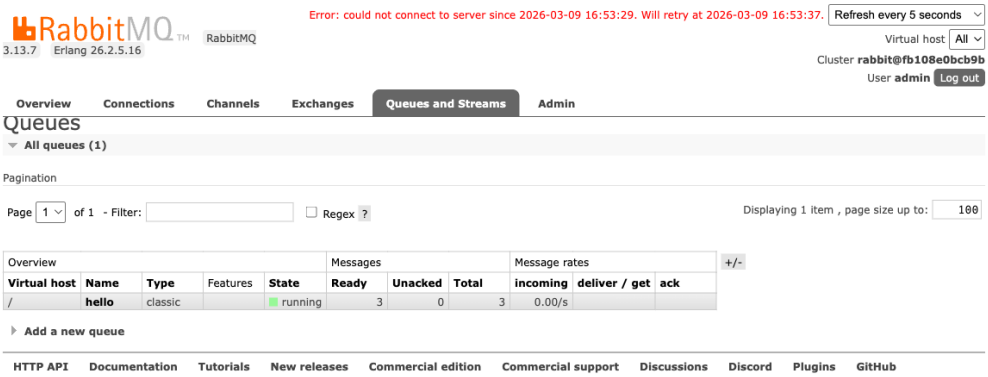
B3: Jalankan publish.py Beberapa Kali, Pantau Dashboard (Q2)

Publisher dijalankan 3 kali, lalu queue *hello* dipantau di dashboard:

```
$ python publish.py
[x] Sent 'Hello RabbitMQ!'

$ python publish.py
[x] Sent 'Hello RabbitMQ!'

$ python publish.py
[x] Sent 'Hello RabbitMQ!'
```



Gambar B3: Queue 'hello' menampilkan 3 pesan Ready

Jawaban Q2:

Setiap kali publish.py dijalankan, counter **Ready** pada queue bertambah 1. Saat consumer aktif, pesan langsung dikonsumsi dan counter turun. Tanpa consumer aktif, pesan terakumulasi di queue (terlihat di kolom Ready). Dashboard juga menampilkan message rate (publish/s dan deliver/s) yang berubah secara real-time.

B4: Dua Consumer Bersamaan (Q3)

Dua consumer dijalankan secara bersamaan; publisher mengirim 6 pesan - setiap consumer menerima tepat 3 pesan (round-robin):

```
# Consumer 1 (Terminal 1):
[*] Waiting for up to 3 messages...
[x] Received Hello RabbitMQ!
[x] Received Hello RabbitMQ!
[x] Received Hello RabbitMQ!

# Consumer 2 (Terminal 2):
[*] Waiting for up to 3 messages...
[x] Received Hello RabbitMQ!
[x] Received Hello RabbitMQ!
[x] Received Hello RabbitMQ!

# Publisher (6x):
[x] Sent 'Hello RabbitMQ!'  (x6)
```

Jawaban Q3:

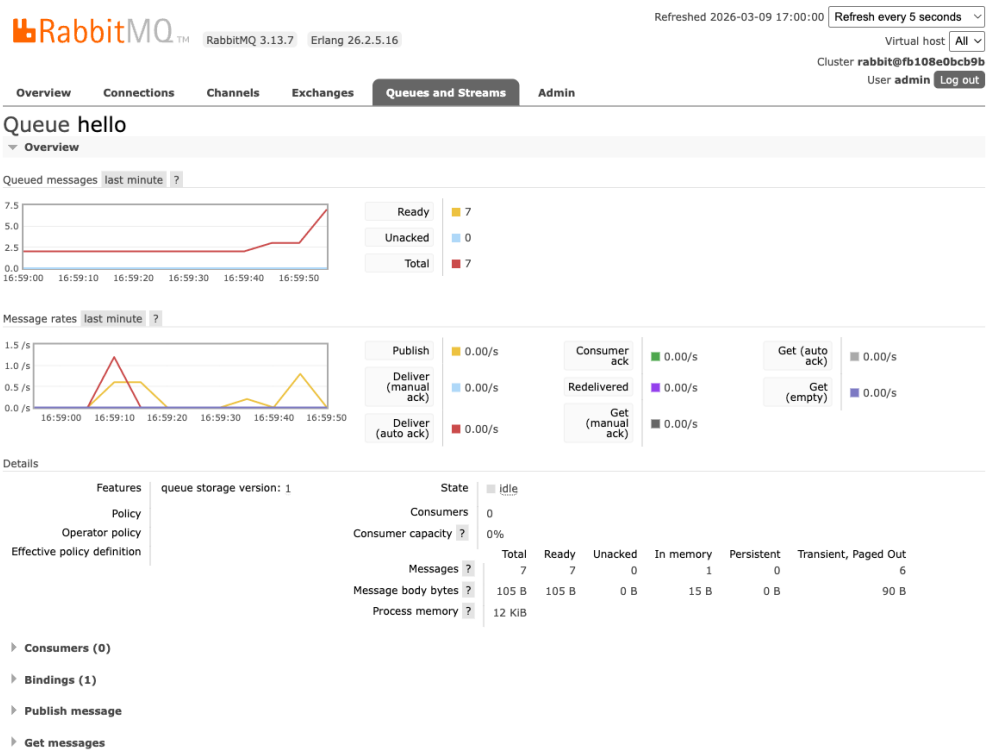
RabbitMQ mendistribusikan pesan menggunakan algoritma **round-robin** antar consumer yang terhubung ke queue yang sama. Dari 6 pesan yang dikirim, Consumer 1 mendapat 3 pesan dan Consumer 2 mendapat 3 pesan - distribusi merata. Setiap pesan hanya diterima oleh SATU consumer (berbeda dengan Kafka tanpa consumer group). Ini ideal untuk scaling horizontal worker processes.

B5: Stop Consumer di Tengah Antrian (Q4)

Queue diisi 5 pesan tanpa consumer aktif, kemudian consumer dihentikan setelah dijalankan. Dashboard menampilkan pesan yang tersisa:

```
# Publish 5 pesan tanpa consumer:
$ python publish.py  # x5
[x] Sent 'Hello RabbitMQ!'  (x5)

# Dashboard menampilkan antrian:
Queue: hello | Ready: 7 | Unacked: 0 | Total: 7 | Consumers: 0
```



Gambar B5: Queue dengan 7 pesan Ready, 0 consumer aktif

Jawaban Q4:

Perilaku bergantung pada setting acknowledgment: (1) Dengan **auto_ack=True** (konfigurasi saat ini): RabbitMQ menganggap pesan sudah diproses begitu dikirim ke consumer. Jika consumer mati setelah menerima pesan tetapi sebelum selesai memprosesnya, pesan HILANG. (2) Dengan **auto_ack=False**: RabbitMQ menunggu ACK eksplisit. Jika consumer mati tanpa mengirim ACK, pesan di-requeue otomatis dan dapat diambil consumer lain. Dashboard menunjukkan pesan tersisa di queue (Ready=7) saat tidak ada consumer aktif -RabbitMQ menyimpan pesan di memory/disk hingga ada consumer yang mengambilnya.

Section C: MySQL Master-Slave Replication

C1: Master dan Slave Berjalan di Port Berbeda (Q1)

```
$ docker ps --filter "name=mysql" \
    --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

NAMES          STATUS          PORTS
mysql-slave    Up 20 minutes   0.0.0.0:3307->3306/tcp
mysql-master   Up 20 minutes   0.0.0.0:3306->3306/tcp

$ docker exec mysql-master mysql -uroot -prootpass -e "SELECT 'master ready';"
master ready

$ docker exec mysql-slave mysql -uroot -prootpass -e "SELECT 'slave ready';"
slave ready
```

Jawaban Q1:

Dua instance MySQL berjalan di container terpisah: **mysql-master** pada port host **3306** dan **mysql-slave** pada port host **3307**. Keduanya dikonfigurasi dengan server-id berbeda (master: server-id=1 dengan binary logging aktif; slave: server-id=2 dengan relay-log). Perbedaan port memungkinkan aplikasi terhubung ke instance yang berbeda dari host yang sama.

C2: Setup Replication

```
# Master IP:
$ docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mysql-master
192.168.240.4

# Pada master - buat user replikasi:
mysql> CREATE USER 'replica'@'%' IDENTIFIED WITH mysql_native_password BY 'password';
mysql> GRANT REPLICATION SLAVE ON *.* TO 'replica'@'%';
mysql> FLUSH PRIVILEGES;
mysql> SHOW MASTER STATUS;
+------------------+----------+
| File             | Position |
+------------------+----------+
| mysql-bin.000004 |      847 |
+------------------+----------+

# Pada slave - konfigurasi replikasi:
mysql> CHANGE MASTER TO
    MASTER_HOST='192.168.240.4', MASTER_USER='replica',
    MASTER_PASSWORD='password', MASTER_PORT=3306,
    MASTER_LOG_FILE='mysql-bin.000004', MASTER_LOG_POS=847;
mysql> START SLAVE;
mysql> SHOW SLAVE STATUS\G
              Slave_IO_Running: Yes
             Slave_SQL_Running: Yes
         Seconds_Behind_Master: 0
                    Last_Error: (none)
```

C3: INSERT di Master, Verifikasi di Slave (Q2)

```
# Pada master:
mysql> CREATE DATABASE testdb;
mysql> USE testdb;
mysql> CREATE TABLE users(id INT, name VARCHAR(50));
mysql> INSERT INTO users VALUES(1,'Test');
mysql> INSERT INTO users VALUES(2,'Alice');
mysql> INSERT INTO users VALUES(3,'Bob');
mysql> SELECT * FROM users;
```

```
+----+-------+
| id | name  |
+----+-------+
|  1 | Test  |
|  2 | Alice |
|  3 | Bob   |
+----+-------+

# Pada slave (setelah insert di master):
mysql> USE testdb;
mysql> SELECT * FROM users;
+----+-------+
| id | name  |
+----+-------+
|  1 | Test  |
|  2 | Alice |
|  3 | Bob   |
+----+-------+
```

Jawaban Q2:

Data yang di-INSERT di master secara otomatis direplikasi ke slave melalui binary log. Proses: (1) Master mencatat setiap perubahan data ke binary log (mysql-bin.000004). (2) Slave IO thread membaca binary log dari master dan menyimpannya sebagai relay log. (3) Slave SQL thread menjalankan event dari relay log ke database slave. Hasilnya identik - ketiga row (Test, Alice, Bob) tersedia di slave tanpa perlu operasi manual apapun.

C4: Force-Stop Master, Cek Slave (Q3)

```
$ docker stop mysql-master
mysql-master

# Slave masih bisa melayani query:
mysql-slave> USE testdb;
mysql-slave> SELECT * FROM users;
+----+-------+
| id | name  |
+----+-------+
|  1 | Test  |
|  2 | Alice |
|  3 | Bob   |
+----+-------+

# Status slave setelah master mati:
mysql-slave> SHOW SLAVE STATUS\G
              Slave_IO_Running: Connecting   ← tidak bisa reach master
             Slave_SQL_Running: Yes          ← data lokal tetap available
         Seconds_Behind_Master: NULL
                 Last_IO_Error: Can't connect to MySQL server on '192.168.240.4:3306' (111)
       Slave_SQL_Running_State: Replica has read all relay log; waiting for more updates
```

Jawaban Q3:

Ketika master dihentikan: (1) Data yang sudah direplikasi ke slave tetap TERSEDIA -slave dapat melayani query read dengan data yang sudah ada (Test, Alice, Bob). (2) IO thread pada slave berstatus *Connecting* - slave terus mencoba terhubung ke master setiap 60 detik. (3) SQL thread tetap aktif (Yes) karena tidak perlu koneksi aktif ke master untuk menjalankan relay log yang sudah ada. Ini menunjukkan slave dapat berfungsi sebagai **read replica** darurat saat master down.

C5: Menulis Langsung ke Slave (Q4)

```
# Master direstart dulu:
$ docker start mysql-master
```

```
# Tulis langsung ke slave:
mysql-slave> USE testdb;
mysql-slave> INSERT INTO users VALUES(99,'DirectOnSlave');
mysql-slave> SELECT * FROM users;
+----+---------------+
| id | name          |
+----+---------------+
|  1 | Test          |
|  2 | Alice         |
|  3 | Bob           |
| 99 | DirectOnSlave |  ← ada di slave
+----+---------------+

# Cek di master - row 99 TIDAK ada:
mysql-master> USE testdb;
mysql-master> SELECT * FROM users;
+----+-------+
| id | name  |
+----+-------+
|  1 | Test  |
|  2 | Alice |
|  3 | Bob   |
+----+-------+
```

Jawaban Q4:

Write langsung ke slave TIDAK direplikasi ke master karena replikasi MySQL bersifat **satu arah (master → slave)**. Row dengan id=99 hanya ada di slave. Risiko: (1) **Data inconsistency** - slave memiliki data yang tidak ada di master, sehingga jika slave dipromosikan menjadi master baru (failover), data ini bisa bertentangan dengan replikasi berikutnya. (2) **Replication break** - jika master kemudian memasukkan row dengan id=99, slave akan error karena duplicate key. Best practice: slave seharusnya bersifat **read-only** (konfigurasi: read_only=1) untuk mencegah penulisan langsung.